LinguGKD: Large Language Model Meets Graph Neural Network in Knowledge Distillation

Chloe Kim
Jan 13, 2026
AAAI-25

Motivation

- Text-Attributed Graphs (TAGs) contain complex semantics
- GNNs struggle with complex language information
- LLMs show promise for TAGs learning, but is expensive for graph inference
  - High computational demands: Billions of parameters
  - Extended latency: Slow inference limiting practical applications

Question

How can we transfer LLM semantic understanding into lightweight GNNs without using LLMs at inference time?

- **Challenge**: Significant architectural differences between LLMs and GNNs
- **Gap**: Knowledge distillation from LLMs to GNNs remains largely unexplored

Goal: Bridging LLMs and GNNs

Solution: LinguGKD

- LinguGKD: **Lingu**istic **G**raph **K**nowledge **D**istillation
- Use LLM as a teacher
  - Instruction tuned LLM generates multi hop semantic embeddings to capture local and global semantics
  - Effectively processes and understands graph-structured data
- User GNN as a student
- Distill local semantic knowledge and global structural information from LLM into GNN

Method

Stage 1: TAG-Oriented Instruction Tuning

- Fine-tune pre-trained LLM with graph-specific prompts
- Creates "LinguGraph LLM" with enhanced graph understanding

Stage 2: Layer-Adaptive Multi-Scale Distillation

- Align LLM and GNN features at multiple granularities
- Transfer knowledge from node-level to graph-level

“LinguGKD outperforms existing graph distillation frameworks, …”

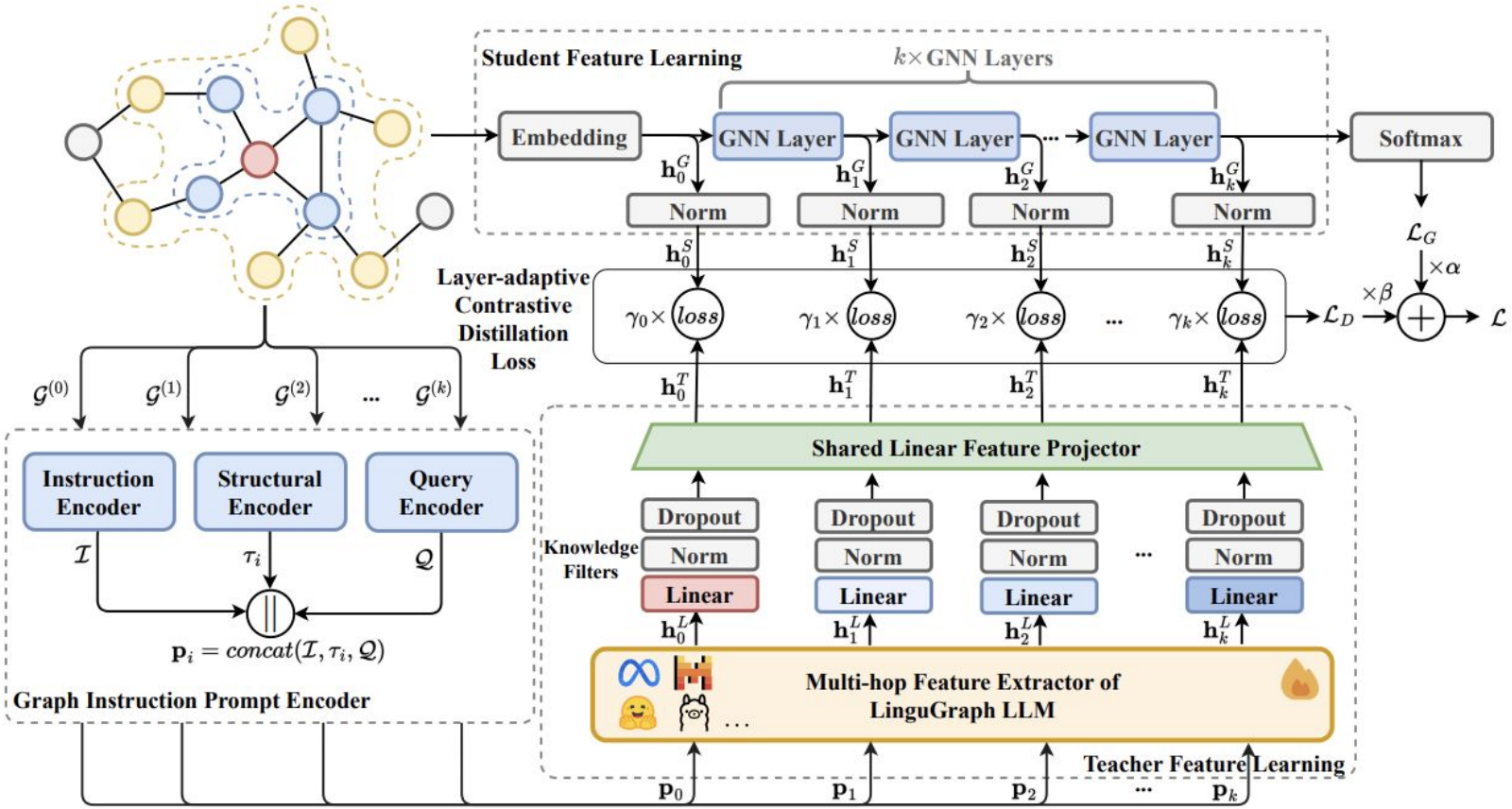


Figure 1: The LinguGKD framework for TAG-oriented LLM-to-GNN knowledge distillation.

Stage 1: TAG-Oriented Instruction Tuning

- For a given node vi, and a max neighbour hop k, construct a series of subgraphs
- For each subgraph, construct instruction prompts with three parts:
  - **Task-Specific Instruction (I)**; defining the graph type:
    - "Classify the node in [graph type], represented as (nodeID, degree, attributes). Categorize into [classification categories] based on [task criterion]"
  - **Structural Description (τI)**; converting the subgraph structure into natural language:
    - "(node [id], [degree], [attrs]) is connected within [l] hops to [l-hop neighbours {in the subgraph}]"
  - **Task-Relevant Query (Q);** posing a task-specific question:
    - "What category should this node ([id], [degree], [attrs]) be classified as?"

Stage 1: TAG-Oriented Instruction Tuning Cont.

- **Prompt** pl = concat(I,τl,Q), where l is…
  - 0 hop: node level semantics
  - 1 hop: neighbourhood semantics
  - Multi hop: global context
- Compute negative log-likelihood loss on text label output
  - E.g., Negative log-likelihood measures how much probability the model gives to the correct label tokens as it generates them
  - Adding up the penalties for not strongly predicting each correct output token
  - Used to fine-tune the teacher LLM into the **LinguGraph LLM**

$$\mathcal{L}_{\text{NLL}}(\mathcal{P}) = -\sum_{l=0}^{k} \sum_{\mathbf{p}_l \in \mathcal{P}_l} \sum_{j=1}^{|y|} \log p(\hat{y}_j | \mathbf{p}_l, \hat{y}_{<j})$$

Stage 2.1: Knowledge Distillation Process

- Teacher (LinguGraph LLM):
  - Extracts semantically-rich node features using instruction prompts
  - Applies neural knowledge filters and layer normalization
- Student (GNN):
  - Extracts multi-hop features through message-passing
  - Compatible with any GNN variant (GCN, GAT, GraphSAGE, GIN)

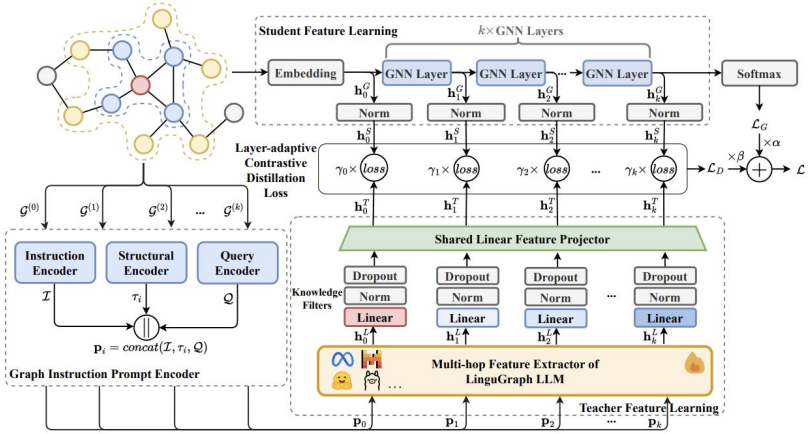


Figure 1: The LinguGKD framework for TAG-oriented LLM-to-GNN knowledge distillation.

Stage 2.1: Knowledge Distillation Process Cont.

- **Teacher (LinguGraph LLM):**
  - Extracts semantically-rich node features using instruction prompts
  - Applies neural knowledge filters and layer normalization

- We feed the full instruction prompt set P into the LLM and use its internal representation as the l-hop node feature.:

$$\mathbf{h}_l^L = \text{Transformer}(\text{Embedding}^L(\mathbf{p}_l); W_{\text{tr}})_{|\mathbf{p}_l|,:}$$

  - Given one instruction prompt, get a vector that represents the node at hop l, according to the LLM.
  - Represents the feature vector corresponding to the last token of the processed instruction prompt after self-attention (how tokens share information)

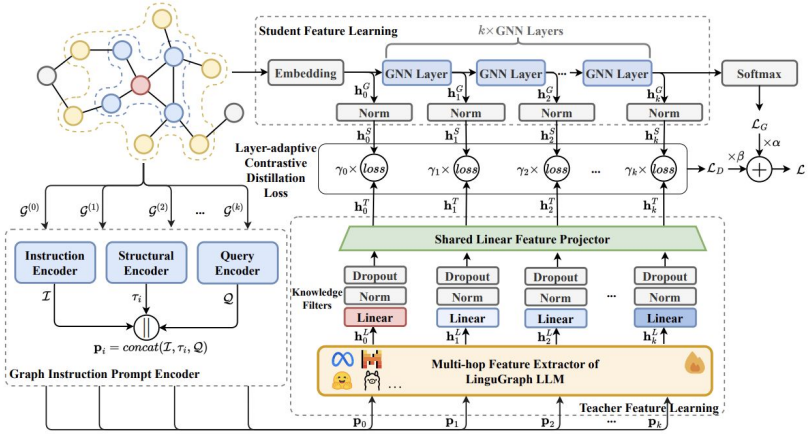


Figure 1: The LinguGKD framework for TAG-oriented LLM-to-GNN knowledge distillation.

Stage 2.1: Knowledge Distillation Process Cont.

- **Teacher (LinguGraph LLM):**
  - Extracts semantically-rich node features using instruction prompts
  - Applies neural knowledge filters and layer normalization

- To prepare LLM-extracted features for knowledge distillation, a two-step processing mechanism is introduced:

$$\mathbf{h}_l^T = \mathcal{M}_p(\text{LayerNorm}(\mathcal{M}_f^l(\mathbf{h}_l^L)); W_p, b_p)$$

  - Takes the **LLM's raw hidden state** (a vector produced inside a neural network while it processes input) for an **l-hop prompt**, **filters** it in a hop-specific way, **normalizes** it, and **projects** it into a shared feature space to produce the teacher node embedding used for distillation
  - Simple explanation: Cleans and reshapes the LLM's output (vectors) so it can be compared with GNN representations.
  - Output: Teacher node embedding at hop l, capturing semantic information at multiple neighbourhood levels.

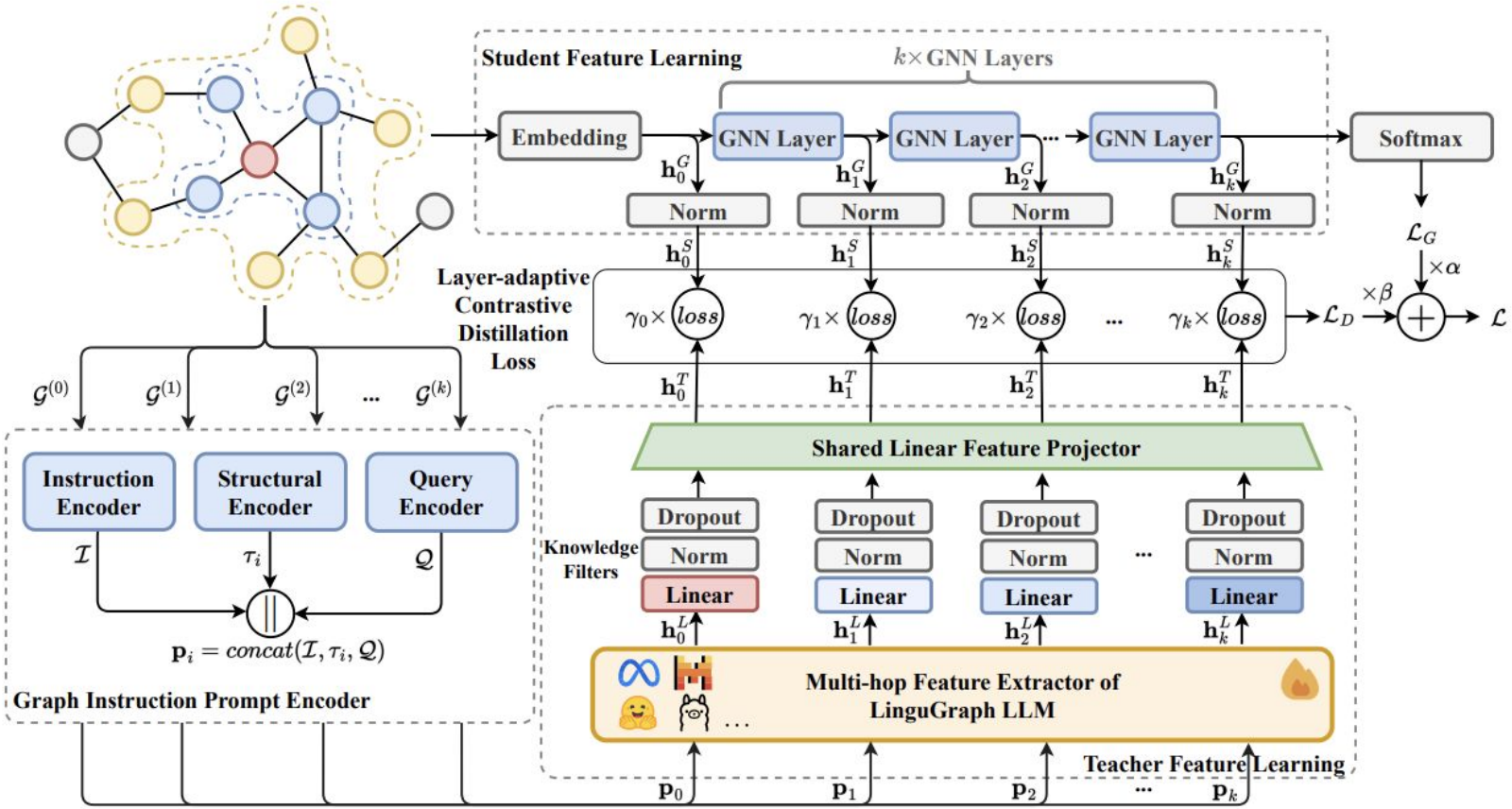


Figure 1: The LinguGKD framework for TAG-oriented LLM-to-GNN knowledge distillation.

Stage 2.1: Knowledge Distillation Process Cont.

- **Student (GNN):**
  - Extracts multi-hop features through message-passing
  - Compatible with any GNN variant (GCN, GAT, GraphSAGE, GIN)
- For a given k-hop neighbour subgraph of a central node vi, the k-order message aggregation is as follows:

$$\mathbf{h}_j^{(0)} = \text{Embedding}^G(x_j), \quad \forall x_j \in \{x_i\} \cup \mathcal{X}_i^{(k)}$$

$$\mathbf{m}_{i \leftarrow j}^{(l)} = \mathcal{M}_{\text{msg}}^{(l)}(\mathbf{h}_i^{(l-1)}, \mathbf{h}_j^{(l-1)}, e_{ij}; W_{\text{msg}}^{(l)})$$

$$\mathbf{h}_l^G = \mathcal{M}_{\text{update}}^{(l)}(\mathbf{h}_i^{(l-1)}, \bigoplus_{v_j \in \mathcal{N}(v_i)} \mathbf{m}_{i \leftarrow j}^{(l)}; W_{\text{update}}^{(l)})$$

  - Shows how a GNN builds node representations layer by layer by initializing node features, exchanging messages with neighbours, and updating each node using aggregated neighbour information
  - Initialize node features from attributes, receive messages from each neighbour, aggregate messages and update the node’s representation

Stage 2.1: Knowledge Distillation Process Cont.

- **Student (GNN):**
  - Extracts multi-hop features through message-passing
  - Compatible with any GNN variant (GCN, GAT, GraphSAGE, GIN)
- To align with the teacher's feature space, a normalization layer has to be applied:

$$\mathbf{h}_l^S = \mathrm{Norm}(\mathbf{h}_l^G), \quad 0 \leq l \leq k$$

  - After each GNN layer, the node's feature vector is normalized so that representations from different layers are on a comparable scale and can be reliably used for learning and distillation
  - Output is a set of hierarchical student node features capturing structural information at multiple scales

Stage 2.2: Layer-Adaptive Multi-Scale Distillation

- **Challenge**: Heterogeneity between LLMs and GNNs
- **Solution**: Learn feature distributions and relationships, not direct values

- Local Distillation Loss: Aligns individual node representations using *contrastive learning*
- Global Alignment Loss: Ensures overall feature distribution matches using KL divergence (measures how much information is lost when one distribution is used to approximate another)
- Layer-Adaptive Mechanism: Dynamically adjusts importance at each layer with trainable parameters

Contrastive Learning

- The model learns by making related examples more alike in the embedding space, while separating examples that are unrelated

- Advantages over direct feature matching:
  - Preserves LLM's semantic structure while adapting to GNN's feature space
  - Learns relative relationships rather than absolute values
  - Enhances discriminative power through hard negative mining

Layer-Adaptive Mechanism

- The layer-adaptive mechanism decides what kind of information each GNN layer should focus on, depending on the dataset

- Automatic adaptation to graph characteristics:
  - Cora Dataset:
    - Higher first-order neighbor factors -> Local topology dominates
      - Nodes are strongly correlated with their immediate neighbours
      - Early GNN layers should focus more on neighbor aggregation, focus on 1-hop structure
  - PubMed Dataset:
    - Elevated structure-free features -> Node attributes more important
      - connectivity alone is less informative
      - Less reliance on local graph structure, more weight on semantic information
- The framework dynamically adjusts importance at each layer based on graph structure

Final loss combines the standard classification loss with the distillation loss that transfers knowledge from the LLM to the GNN; Softmax turns GNN's output scores into class probabilities.

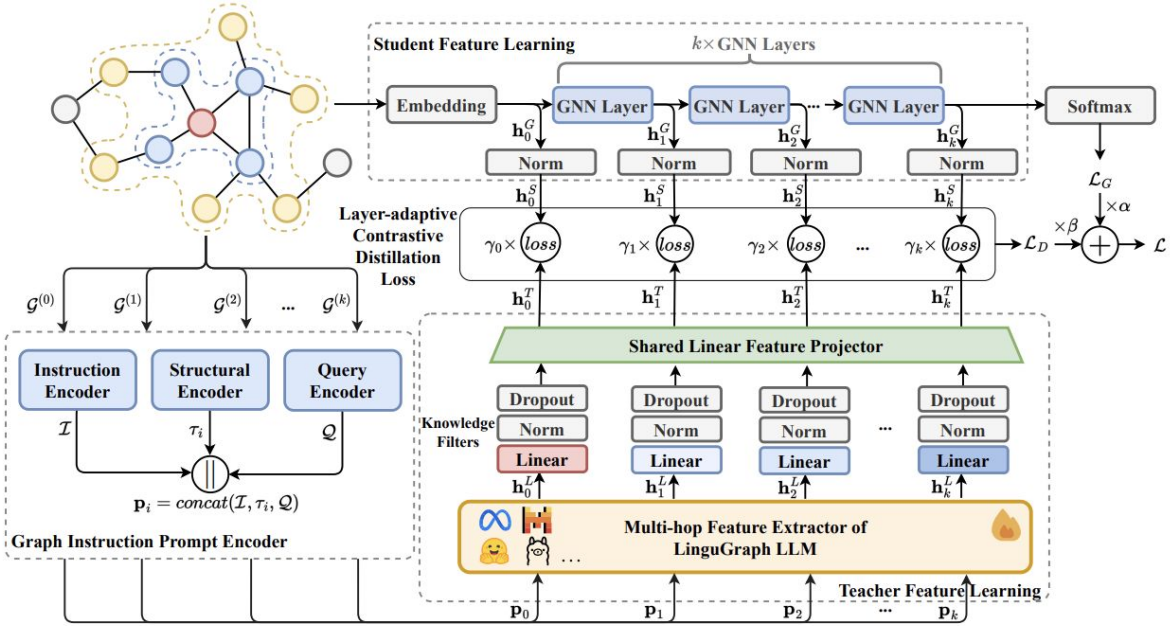


Figure 1: The LinguGKD framework for TAG-oriented LLM-to-GNN knowledge distillation.

Datasets

- Academic papers as nodes and citations as edges
- Teachers (LLMs): Llama2-7B, Llama3-8B
- Students (GNNs): GCN, GAT, GraphSAGE, GIN

	Cora	**PubMed**	**Arxiv**
# Node	2,708	19,717	169,343
# Edge	5,429	44,338	1,166,243
# Class	7	3	40
# Features	1433	500	128
Embedding Tech.	BoW	TF-IDF	Skip-gram
$\lvert\mathcal{D}_{tr}\rvert : \lvert\mathcal{D}_{val}\rvert : \lvert\mathcal{D}_{test}\rvert$	6:2:2	6:2:2	5.4:1.8:2.8

Table 1: Dataset Statistics

Main Results: Performance of LinguGraph LLMs (RQ1)

RQ1: How do LinguGraph LLMs compare to existing graph learning baselines?

LinguGraph LLMs consistently outperforms existing baselines across all datasets

- Supports the potential of LLMs as foundational models for graph learning, establishling that LinguGraph LLMs as strong teacher models for the KD framework.

Methods	Cora		PubMed		Methods	Arxiv	
	Acc.↑	F1↑	Acc.↑	F1↑		Acc.↑	F1↑
GCN	86.53±0.92	85.66±0.78	86.12±0.93	85.64±0.82	GCN	71.74±0.21	71.04±0.37
GAT	86.12±0.95	85.05±0.88	85.49±0.76	84.89±0.71	GAT	73.66±0.33	72.44±0.19
GraphSAGE	87.08±0.85	85.96±0.73	87.69±0.92	87.38±0.68	GraphSAGE	71.19±0.26	70.87±0.45
GIN	86.60±0.91	85.37±0.74	85.84±0.92	85.31±0.63	GIN	71.62±0.47	71.13±0.33
BernNet (He et al. 2021)	88.52±0.95	87.96±0.85	88.48±0.41	87.52±0.79	GTAN (Wu and Wang 2022)	72.97±0.17	71.77±0.22
FAGCN (Bo et al. 2021)	88.85±1.36	87.92±0.65	89.98±0.54	88.72±0.53	UniMP (Shi et al. 2021)	73.11±0.20	72.14±0.38
GCNII (Chen et al. 2020)	88.93±1.37	87.58±0.71	89.80±0.30	88.96±0.62	GCNII (Chen et al. 2020)	72.74±0.00	72.22±0.44
RevGAT (Li et al. 2021)	89.11±0.00	87.65±0.58	88.50±0.05	87.12±0.73	RevGAT (Li et al. 2021)	74.02±0.18	73.56±0.29
ACM-GCN+ (Luan et al. 2022)	89.75±1.16	88.94±0.54	90.96±0.62	89.77±0.51	E2EG (Dinh et al. 2023)	73.62±0.14	72.96±0.26
Graphormer (Ying et al. 2021)	80.41±0.30	79.98±0.56	88.24±1.50	87.52±0.71	SGFormer (Wu et al. 2024)	72.63±0.13	71.58±0.42
LinguGraph-Llama2 (7B)	88.19±0.83	88.12±0.73	94.09±0.78	93.55±0.61	LinguGraph-Llama2 (7B)	75.67±0.52	75.60±0.41
LinguGraph-Llama3 (8B)	91.51±0.46	**91.53±0.18**	**95.59±0.29**	**95.55±0.10**	LinguGraph-Llama3 (8B)	**79.73±0.18**	**79.29±0.56**
$GCN_{(Llama2)}$	90.59±0.71	89.62±0.66	88.97±0.82	88.56±0.71	$GCN_{(Llama2)}$	73.87±0.22	73.87±0.61
$GCN_{(Llama3)}$	90.77±0.28	90.35±0.37	89.76±0.44	89.46±0.37	$GCN_{(Llama3)}$	74.68±0.45	74.29±0.32
$GAT_{(Llama2)}$	90.33±0.67	89.72±0.59	87.93±0.28	87.42±0.36	$GAT_{(Llama2)}$	74.92±0.14	74.48±0.28
$GAT_{(Llama3)}$	91.51±0.35	91.45±0.58	88.31±0.76	87.93±0.65	$GAT_{(Llama3)}$	75.71±0.41	75.06±0.36
$GraphSAGE_{(Llama2)}$	90.22±0.77	89.89±0.19	89.96±0.50	89.67±0.34	$GraphSAGE_{(Llama2)}$	72.53±0.61	72.42±0.49
$GraphSAGE_{(Llama3)}$	**91.70±0.51**	91.08±0.62	90.14±0.56	89.96±0.48	$GraphSAGE_{(Llama3)}$	75.38±0.38	75.22±0.32
$GIN_{(Llama2)}$	90.26±0.67	89.20±0.48	87.73±0.29	87.20±0.30	$GIN_{(Llama2)}$	73.71±0.25	73.42±0.10
$GIN_{(Llama3)}$	91.33±0.28	91.05±0.53	89.22±0.79	88.87±0.61	$GIN_{(Llama3)}$	75.64±0.46	75.28±0.39
Avg. Dist. Gains	4.61%	5.22%	2.79%	2.84%	*Avg. Dist. Gains*	3.85%	4.22%

Table 2: Node classification performance of various graph learning models across selected datasets, with the highest-performing outcomes in **bold**, second-best scores highlighted in gray, and best baseline performances underlined. The prefix *LinguGraph*– represents teacher LLMs obtained by fine-tuning different PLMs with graph instruction prompts, while GNNs with different subscripts represent student models distilled from the corresponding teacher LLMs indicated by the subscripts. The term *Avg. Dist. Gains.* refers to the average knowledge distillation gains obtained by different student GNNs.

Distilled GNN Performance (RQ2)

RQ2: What performance gains do distilled GNNs achieve?

Significant improvements across all student GNNs. The average distillation gains range from 2.79% (PubMed) to 4.61% (Cora):

Other KD Frameworks (RQ3)

RQ3: How does LinguGKD compare to other graph knowledge distillation frameworks?

Improvements: +3.06% (Cora),

+3.04% (Arxiv) over best baselines

Model	Cora	Arxiv
Teacher of baselines	88.93	73.91
Teacher of LinguGKD	**91.51**	**79.73**
KD (Hinton, Vinyals, and Dean 2015)	86.38	71.55
FitNet (Romero et al. 2015)	85.40	71.38
LSP (Yang et al. 2020)	84.92	71.52
GraphAKD (He et al. 2022)	86.39	-
G-CRD (Joshi et al. 2024)	-	71.64
LinguGKD	90.77	74.68

Table 3: Results (%) of different knowledge distillation methods. For baselines, GCNII is used as the teacher on Cora, and GAT on Arxiv. LinguGKD uses LinguGraph-Llama3 as the teacher. All methods use GCN as the student model. '-' means not available.

Efficiency Trade-offs (RQ4)

RQ4: What are the trade-offs between LinguGraph LLMs and distilled GNNs?

- LinguGraph LLMs:
  - 6.74B-8.03B parameters, >25GB storage
  - Inference time: >0.5 seconds
- Distilled GNNs:
  - 7.78M-11.97M parameters, 0.03-0.04GB storage
  - Inference time: <0.1 seconds

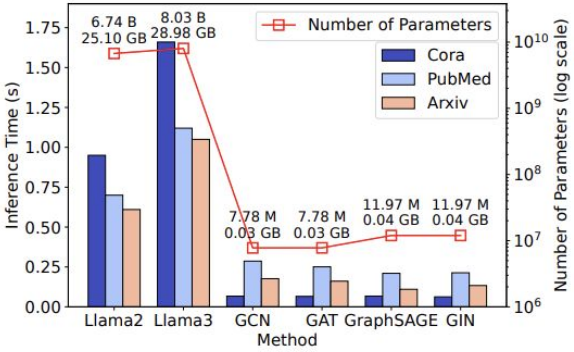


Figure 2: Model size and inference time of different models.

- Smaller, faster, with comparable accuracy
- Makes distilled GNNs ideal for resource-constrained or real-time applications

Hyperparameter Impact (RQ5)

RQ5: How do different hyperparameters affect the performance of LinguGKD?

- Neighbour Orders (k-hop):
  - Performance improves up to 2-hop
  - Beyond 2-hop: over-smoothing in vanilla GNNs
  - Distilled GNNs mitigate over-smoothing effects
- Hidden Feature Dimensions(how many numerical features are used to represent each node internally in the model):
  - Vanilla GNNs plateau at 128 dimensions
  - Distilled GNNs improve continuously up to 1024 dimensions
- Optimal: 2-hop neighbours, 1024-dimensional features

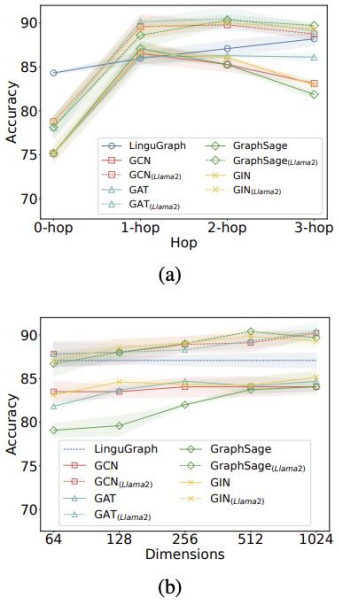


Figure 3: The results of performance impacts as the variations of neighbor orders and hidden feature dimensions.

Ablation Study: Graph Instruction Tuning

Graph instruction tuning is crucial as it's shown to improve LLM accuracy from 8-30% (initial pre-tuned LLMs) to 75-95%

Methods	Cora		PubMed		Arxiv	
	T.	S.	T.	S.	T.	S.
Llama2	9.23	85.21	25.04	85.88	10.72	70.94
LinguGraph-Llama2	88.19	90.59	94.09	88.97	75.67	73.87
Llama3	14.92	85.48	30.17	86.06	15.31	70.58
LinguGraph-Llama3	**91.51**	**90.77**	**95.59**	**89.76**	**79.73**	**74.68**

Table 4: Node classification accuracy (%) of teacher LLMs (T.) and their distilled student GNNs (S.) before and after graph instruction tuning across different datasets.

Ablation Study: Effectiveness of Layer-Adaptive Multi-scale Contrastive Distillation

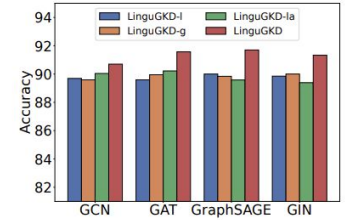
Three variants:

- LinguGKD-l: Without local distillation loss
- LinguGKD-g: Without global alignment loss
- LinguGKD-la: Without layer-adaptive mechanism

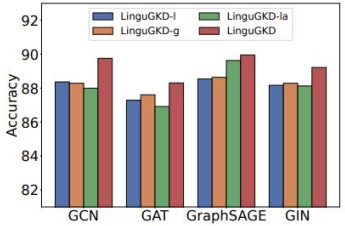
Complete LinguGKD framework consistently outperforms all variants

- Removing any component leads to suboptimal knowledge transfer
- Layer-adaptive mechanism crucial for capturing

  multi-scale graph representations

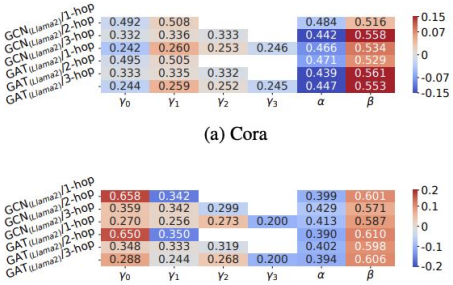


(a) Accuracy on Cora



(b) Accuracy on PubMed

Figure 4: Performance comparison of GNNs distilled with LinguGKD and the ablated variants.



(a) Cora

(b) PubMed

Figure 5: Heatmap visualization of layer-adaptive factors (γ_l) and classification-distillation loss weights (α, β) during training on Cora and PubMed datasets. Darker colors indicate higher values relative to the average.

Limitations & Future Work

Current Limitations:

- Limited to text-attributed graphs (node attributes available)
- Computational cost of initial LLM fine-tuning (graph-to-text and instruction tuning)
- Not task specific (like Fraud Detection in our case)
- Scalability concerns for large or dynamic graphs

Future Work:

- Extend to graphs without native node text
- Apply to task-specific settings (e.g., fraud, anomaly detection)
- Evaluate across more datasets and graph settings

Conclusion

- LinguGKD effectively bridges semantic understanding of LLMs with efficiency of GNNs
- Achieves significant performance improvements:
  - Average 3-5% accuracy gains across all GNN architectures
  - Distilled GNNs match or exceed complex baselines
- Provides practical solution for resource-constrained deployment
- Establishes promising direction for leveraging LLM advances for GNN improvement

Therefore enables advanced graph learning in real-world scenarios previously limited by computational constraints